



NXP Face Recognition Solution for MCUs and Crossovers Model Customizations

Vision processing component of Video Doorbell Solution



Contents

1. Description.....	4
2. Current models being used	6
a) Face detection – SCRFD-KPS.....	6
i. Model comparison	6
ii. Model performance.....	6
b) Antispoofing – MiniXception.....	7
i. Model comparison	7
ii. Model performance.....	7
c) Face Recognition – MobileFaceNet	8
i. Model comparison	8
ii. Model performance.....	8
3. How to change models	10
a) How to integrate with eIQ-neutron-sdk. Error! Bookmark not defined.	
b) How to replace current models in pipeline with a new one.....	11
4. How to retrain current models with new dataset	14
a) Retrain Antispoofing model.....	14
i. Base Architecture.....	14
ii. Dataset.....	14
iii. Training modes.....	15
iv. Command-line arguments:	15
v. Evaluation Metrics.....	18
vi. TFLite Conversion Features	18
vii. Annotation particularities.....	19
b) SCRFD.....	19
i. Model retraining.....	19
ii. Annotation particularities.....	19



c)	MobileFaceNet Face Recognition Model.....	19
i.	Model Overview	19
ii.	Dataset Requirements.....	21
iii.	Training Modes.....	22
iv.	Command-Line Argument.....	22
v.	Training Examples.....	26
vi.	Evaluation Metrics.....	27
vii.	Annotation particularities.....	27
5.	References.....	28



1. Description

The purpose of the face recognition solution is to provide an ML-enabled framework that matches one's face to one's identity; this can facilitate use-cases such as secure access and others.

The solution itself is built around NXP MCU Vision Pipeline library (also known as Multimedia Processing Pipeline, MPP) a middleware available in NXP's MCUX SDK. The processing pipeline is built using different processing elements, from the camera(s), multiple image processing stages, inference engine that runs multiple ML models and display elements.

The purpose of this present document is to describe the ML models used and how to customize them (re-train or replace) the models used and how one can retrain or fully change the ML models.

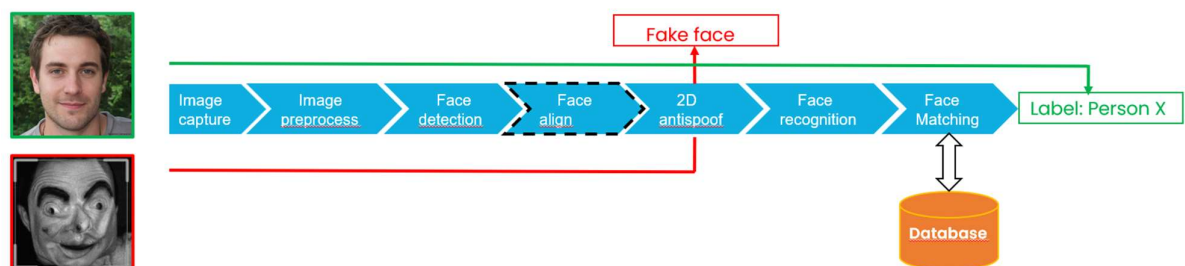


Figure 1 Face recognition pipeline

Inference stages shortly explained

- **Face Detection**
 - Find the faces in the input video, determine the coordinates of its bounding box, and crop the image accordingly.
- **Face Alignment**(optional):
 - Fixes the alignment if the face is not centered and aligned vertically. Face alignment/centering normalizes pose and position so the network sees consistent facial geometry.



- Many techniques can be used like aligning the face based on the eyes location, face landmark estimation, etc.
- If the feature is disabled an alignment check must be applied to filter out faces rotated or tilted beyond acceptable.
- **2D anti spoofing:**
 - Face recognition systems are widely used in different areas. However, they are vulnerable to spoofing attacks, where fake biometric data is used to deceive the system.
 - Therefore, face anti-spoofing is quite vital to the security of face recognition systems.
 - Examples of spoof attacks: digital display of recorded face, masks (latex, paper, etc.)
- **Face recognition:**
 - This is the most important step of the pipeline where the face features are extracted
 - Each individual has unique features; this step of the pipeline converts these facial features into a numerical representation called face embedding.
- **Face Matching:**
 - Check whether the computed embedding of the detected face matches an entry in the database of recorded faces.
 - Many approaches can be used like Euclidean distance, cosine similarity etc.



2. Current models being used

a) Face detection – SCRFD-KPS

i. Model overview

Current solution:

SCRFD-KPS is a lightweight face detection and landmarks model designed for edge computing devices. The model has been tested on RGB data (LFW and Wider face) and few samples of the IR internal data. The model accuracy has **not yet** been evaluated on RGB (<https://github.com/deepinsight/insightface/blob/master/detection/scrfd/README.md>).

ii. Model performance

SCRFD uses a ResNet-based backbone with a PAFPN neck and a lightweight convolutional head, producing multi-scale feature maps for efficient face detection; the model outputs classification scores, bounding boxes, and 5 facial keypoints (eyes, nose, mouth corners) for every detected face, enabling robust landmark-aware detection across different scales.

Dataset	AP Value	NME Value
Wider Face	0.9948	0.1366
LFW	0.9993	0.2586

Table 1 SCRFD performance per Dataset

- Average Precision (AP) – AP evaluates how well the INT8 quantized model's bounding boxes match the FP32 model's bounding boxes, which are treated as the reference or "ground truth". Average Precision is a metric used to evaluate the performance of ranking systems or object detection models. It summarizes the precision-recall curve into a single value. AP ranges from 0 to 1, where 1 indicates perfect detection (all predicted boxes match the



reference with high confidence) and 0 means no useful detections
—> higher values are better.

- Normalized Mean Error (NME) - NME evaluates the accuracy of facial landmark predictions comparing the positions of key facial points (like eyes, nose, mouth) between the INT8 quantized and FP32 models. NME measures how far off predictions are from the true values, meaning that the values that are closer to 0 indicate a better performance of the model.

b) Antispoofing – MiniXception

iii. Model comparison

NXP created a custom model, which architecture is based **on MiniXception adapted for an AntiSpoofing task**, fully quantized TFlite model, with an input resolution of 96*96.

iv. Model performance

A typical Face Antispoofing model built on an Inception-style architecture leverages multi-scale convolutional branches to distinguish real faces from spoofing attacks; like other deep learning anti-spoofing systems, it operates as a binary classifier.

Total fake images count: 6467

- liveness fake 0.9949

Total real images count: 7407

- liveness real 0.9620

- accuracy: 0.9776

- correct num: [6428. 6943.]

- blurry num: [6. 190.]

- liveness num: [6467. 7407.]



c) Face Recognition – MobileFaceNet

v. Model comparison

For **RGB Face Recognition** we explored the following architectures:

- **GhostFaceNet, ArcFace and edge face:** a huge loss in accuracy after fully quantization
- **DeepID:** accuracy and model size constraints.
- **QuantFace** : good tradeoff accuracy/size but has license issues

For the **InfraRed Face Recognition**, no opensource models are available.

Current solution

MobileFacenet RGB and IR: fully quantized TFlite models, with an input resolution of 96*96.

<https://www.kaggle.com/code/jasonhchwong/mobilefacenet/>

vi. Model performance

MobileFaceNet follows a compact convolutional architecture built from lightweight building blocks such as Conv2D + BatchNorm + PReLU blocks, depthwise separable convolutions, and residual depthwise blocks; the network begins with a standard convolution stem, progresses through stacked depthwise-residual units at increasing channel widths, and ends with a 1×1 convolution, a global depthwise 6×6 convolution, flattening, a linear embedding layer, and an L2-normalization step to produce the final face embedding.

	Flip	Face align	Face detect	Accuracy	TAR@FAR=0.01	TAR@FAR=0.001	TAR@FAR=0.0001	TAR@FAR=0.00001
Per tensor quantized	Y	Y	Y	0.99130 Thresh:1.38 65.5%	0.96901 Thresh:1.603 59.925%	0.941755 Thresh:1.488 62.8%	0.920711 Thresh:1.382 65.45%	0.82522 Thresh:1.283 67.925%

Table 2 MobileFaceNet performance



- **Evaluation data:** 10.000.000 **negative** pairs and 100.000 **positive** pairs from the LFW dataset.
- **Similarity metrics:** The **squared Euclidian distance** has been used; thresholds are computed based on this metric only.



3. How to replace one or multiple models of the pipeline

d) eIQ Neutron SDK Overview

[eIQ Neutron SDK](#) is a software package that includes the Neutron Converter tool and eIQ Neutron libraries to run Neutron converted neural network models on devices that have an eIQ Neutron NPU like [MCX N](#), [i.MX RT700](#), or [i.MX95](#). It contains tools such as:

- Quantizer
- Extractor
- Converter

Requirements:

- Neutron SW 2.2.2 used for SDK 25.12.
- Post-training quantized models in .tflite format.

NOTE: If post-training quantization was not performed prior to TFlite format conversion, please refer to the **TFLiteQuantizerUserGuide** and **TFLiteProfilerUserGuide** documentation.

Steps:

- To setup the eiq-neutron-sdk-2.2.2 refer to **NeutronConverterUserGuide**, this will ensure that all the tools available inside the sdk (quantizer, extractor, converter) are working properly.
- After quantization the converter can be used to convert models in TFlite format for execution on the Neutron NPU using the command:

```
./neutron-converter --input <model> --target  
imxrt700'
```

NOTE: Some operators are not currently supported for conversion, for the list of supported operators refer to **SupportedOperatorsC**. In case of performance mismatches, check **model_all_ops_micro.cpp** (inside mpp) to see which operators have a CPU fallback.



e) How to replace current models in pipeline (model conversion)

- The steps followed for making the model compatible with the Face Recognition Application, before integration:
 - Extraction from the .tflite flat buffer – only for MobileFaceNet because the final graph includes the L2 normalization, which in our case is done offline.

```
'./tflite-extractor --input <path_to_tflite_model> --  
output-tensors <name_of_output_tensor> --output  
<output_path_to_extracted_tflite_model>'
```

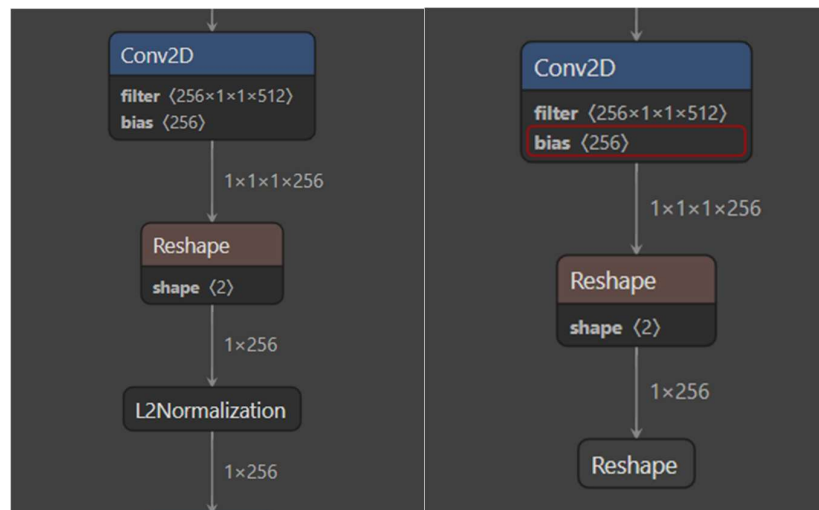


Figure 2 Graph structure after extraction

NOTE: Because most MobileFaceNet models apply an L2-normalization layer to their embeddings, this step must be removed after export (refer to the Face Recognition document, **elQ-neutron-sdk extractor**), as normalization is already performed offline and the similarity metric relies on squared Euclidean distance between feature vectors. The ArcFace layer will also be stripped away when exporting to a .tflite format.

If the .tflite model is not fully quantized, then you can use the profiler and quantizer:

- serialize samples for graph profiling



```
'cd scripts
python serialize_image.py -input-path
<path_to_train_samples> -output-path
<output_directory_path_for_binary_samples> -output-
format bin -output-shape <image_resolution> -output-
type float32 -output-mode 0tol -output-layout
<channel_order> -output-channel-order RGB'
```

2. extract information about every tensor, the quantization data should contain around 1000 sample from the train set - this step generates a CSV file that will be used by the quantizer tool

```
 './tflite-profiler --input <path_to_tflite_model> --
dataset
<name_of_input_tensor>,<serialized_image_samples_from
train_data> --output <path_to_csv_file>'
```

3. quantize using the tensor information generated CSV file

```
 './tflite-quantizer --input <path_to_tflite_model> --
profile <path_to_csv_file> --output
<path_to_quantized_model> --verbose True --
quantization-constant-type INT8'
```

- The quantized and eIQ Neutron converted TensorFlow Lite model must now be processed to be used in the Face Recognition SW Application. This step converts the TensorFlow Lite in a C header file that will be compiled with the application.
'xxd -i model.tflite > model.h'
- Based on the graph, the headers contain information about the models such as width, height and color format. Other parameters include input mean and std, output scale and zero point the number of channels, and the maximum number of boxes; as well as recognition threshold used by MobileFaceNet.

NOTE: Converting Non TensorFlow models. Following model retraining process if the framework used is not TensorFlow based, the model can then be converted to TFLite format using the following git repository [<link>](#).

NOTE: The size of the Tensor Arena buffer for TensorFlow Lite-Micro may change depending of the Converter version and needs to be adjusted accordingly (check the **mpp_config.h**



file for additional information). The size of necessary tensor arena is calculated once the model has been converted for the NPU.



4. How to retrain current models using a new dataset

One may wish to use existing models but would like to use private data or images that are licensed for commercial use specific to one's industry to retrain the models to better fit the end user product.

This chapter addresses this topic, describing how to re-train each of the models used in the Face Recognition Solution with new data.

f) Retrain Antispoofing model

i. Base Architecture

- Type: Lightweight Convolutional Neural Network (CNN) with Residual Connections
- Inspiration: The architecture is inspired by MobileNet and Xception architectures, optimized for mobile/edge deployment
- Key Feature: Uses Separable Convolutions (depth-wise separable convolutions) to reduce model size and computational cost

ii. Dataset

The dataset should be organized in the following structure

```
dataset_root/
├── folder1/
│   ├── live/
│   │   ├── image1.jpg
│   │   ├── image1.txt # Optional: bounding box coordinates
│   │   └── ...
│   └── spoof/
│       ├── image1.jpg
│       ├── image1.txt # Optional: bounding box coordinates
│       └── ...
├── folder2/
│   ├── live/
│   └── spoof/
└── ...
```



- Label convention:
 - live/ folder: Real faces (label = 1)
 - spoof/ folder: Fake/spoofed faces (label = 0)

iii. Training modes

- Generator mode (recommended):
 - Memory-efficient: Loads images on-the-fly during training
 - Best for: Large datasets that don't fit in RAM
 - Default mode: Enabled by default.
- Legacy mode:
 - Load-all: Loads entire dataset into memory
 - Best for: Small datasets, faster training if RAM permits
 - Enable with: `--legacy_mode` flag

iv. Command-line arguments:

- Dataset arguments:
 - **`--dataset_root`** (string, required)
Root directory of the dataset containing the training data.
 - **`--train_folders`** (list of strings, optional)
Specific folders to use for training. If not specified, all available folders in the dataset root will be used.
 - **`--grayscale`** (flag, default: disabled)
Convert images to grayscale during loading. Enable this flag for single-channel (grayscale) model training.
 - **`--use_bbox`** (flag, default: enabled)
Use bounding boxes for face cropping. Requires corresponding .txt files with bounding box coordinates for each image.
- Training mode arguments:



- **--use_generator** (flag, default: enabled)
Use memory-efficient generator mode that loads images on-the-fly during training. This is the recommended mode for large datasets.
- **--legacy_mode** (flag, default: disabled)
Use legacy load-all mode that loads the entire dataset into memory before training. Only recommended for small datasets that fit comfortably in RAM.
- Multi-configuration arguments:
 - **--sample_sizes** (list of integers, default: 2000 5000 10000)
List of dataset sizes to test in separate training runs. The script will train one model for each sample size, allowing comparison of performance across dataset scales.
- Model Arguments
 - **--input_shape** (three integers, default: 96 96 1)
Input shape for the model specified as height, width, and channels. Use 96 96 1 for grayscale or 96 96 3 for RGB images.
- Training Hyperparameters
 - **--optimizer** (string, default: adam)
Optimizer to use for training. Choices: adam or sgd.
 - **--learning_rate** (float, default: 0.0001)
Initial learning rate for the optimizer. This value may be automatically reduced during training by the ReduceLROnPlateau callback.
 - **--momentum** (float, default: 0.9)
Momentum value for SGD optimizer. Only used when --optimizer sgd is specified.
 - **--batch_size** (integer, default: 32)
Number of samples per training batch. Larger values may speed up training but require more GPU/CPU memory.



- **--epochs** (integer, default: 10)
Maximum number of training epochs. Training may stop earlier if early stopping is triggered.
- **--validation_split** (float, default: 0.2)
Fraction of data to use for validation (range: 0.0 to 1.0). For example, 0.2 means 20% validation and 80% training.
- Callback Arguments
 - **--patience** (integer, default: 50)
Number of epochs with no improvement on validation loss before training is stopped (early stopping patience).
 - **--reduce_lr_patience** (integer, default: 5)
Number of epochs with no improvement before reducing the learning rate by a factor of 0.5.
 - Output Arguments
 - **--output_dir** (string, default: ./output_multi)
Base directory where all training outputs, models, and logs will be saved.
 - **--experiment_prefix** (string, default: antispoofing)
Prefix used for naming experiment directories. Each experiment will be named as {prefix}_{sample_size}samples.
- TFLite Conversion
 - **--convert_tflite** (flag, default: disabled)
Convert the trained Keras model to TensorFlow Lite format after training. The TFLite model will be saved alongside the Keras model.
 - **--quantize** (flag, default: disabled)
Apply full INT8 quantization when converting to TFLite format. This significantly reduces model size and improves inference speed on mobile devices. Only takes effect when --convert_tflite is also enabled.



- Reproducibility
 - **--random_state** (integer, default: 42)
Random seed for reproducibility. Using the same seed ensures consistent data splits and initialization across multiple training runs.

v. Evaluation Metrics

The training script computes the following metrics:

- Accuracy: Overall classification accuracy
- FPR (False Positive Rate): Spoof incorrectly classified as Live
- FNR (False Negative Rate): Live incorrectly classified as Spoof
- ACER (Average Classification Error Rate): $(FPR + FNR) / 2$

Lower ACER is better – it is the primary metric for anti-spoofing performance.

vi. TFLite Conversion Features

1. Fixed Batch Size Support

- Feature: Can convert with `fixed_batch_size=1` for optimized single-image inference
- Benefit: Better optimization for edge deployment scenarios

2. Full Integer Quantization (INT8)

- Implementation: Post-training quantization with representative dataset
- Input/Output Types: Converted to uint8 for maximum efficiency
- Supported Ops: `TFLITE_BUILTINS_INT8` for hardware acceleration
- Size Reduction: ~4x smaller than float32 model
- Speed Improvement: 2-4x faster on mobile CPUs with NEON/SIMD support

3. Representative Dataset Calibration

- Purpose: Determines optimal quantization ranges for each layer



- Default: Uses 100 random samples from training data
- Customizable: Can provide custom representative dataset for better accuracy
- Performance Characteristics

vii. Annotation particularities

- Both train and test data have been collected by NXP.
- The training data should be split into 2 folders: one that contains the real data and one that contains the spoof data.

g) Retrain SCRFD

i. Model retraining.

- For retraining this model, please use the instructions found in this [README](#)
- As it's a PyTorch based model please convert it to TFLite format using the following git repository [<link>](#).

ii. Annotation particularities

- WIDERFace provides **393k annotated faces across 32k images**, covering large variations in scale, pose, and occlusion.
- It is split into **Easy, Medium, and Hard** difficulty subsets for evaluation.
- Annotations include **bounding boxes** and supporting **five key facial landmarks** for models like SCRFD-KPS.

h) Retrain MobileFaceNet Face Recognition Model

i. Model Overview

Base Architecture:

- Type: Lightweight Convolutional Neural Network with Inverted Residual Blocks



- Inspiration: Based on MobileNetV2 architecture, optimized for mobile/edge deployment
- Key Features:
 - Uses Depthwise Separable Convolutions to reduce model size and computational cost
 - Inverted residual blocks with linear bottlenecks for efficient feature extraction
 - Global depthwise convolution for spatial feature aggregation
 - L2-normalized embeddings for metric learning
- Model Size: ~1MB (INT8 fully quantized), ~3MB (Float32)
- Embedding Dimension: 256-dimensional face feature vector

NOTE: The size of the embedding vector can be changed by adjusting the number of filters of the last linear 1x1 convolution:

```
x = Conv2D(256, (1, 1), padding='same', strides=(1, 1),  
          kernel_initializer='glorot_uniform',  
          kernel_regularizer=regularizers.l2(weight_decay),  
          use_bias=False, name='conv5')(x)
```

This can vary from 128 to 512 for the final feature vector.
Input Size: 96×96 or 112×112 RGB images

NOTE: By changing the resolution of the input image to 112x112, the spatial size of the depthwise convolution kernels must be changed since the output feature map also suffers modifications.

e.g. `x = DepthwiseConv2D((6, 6), ...)` → `x = DepthwiseConv2D((7, 7), ...)`

Loss Function:

- Type: ArcFace (Additive Angular Margin Loss)
- Purpose: Learns discriminative face embeddings by adding angular margin penalty
- Parameters:
 - Scale (s): 64.0
 - Margin (m): 0.50 radians ($\approx 28.6^\circ$)



ii. Dataset Requirements

Dataset Format

The training script expects [TFRecord](#) format with the following structure:

```
features = {  
    'image_raw': tf.io.FixedLenFeature([], tf.string), # JPEG-encoded image  
    'label': tf.io.FixedLenFeature([], tf.int64)      # Integer class label  
}
```

File Organization

```
dataset_directory/  
├── faces_ms1m_refine_v2_112x112-00000.tfrecord  
├── faces_ms1m_refine_v2_112x112-00001.tfrecord  
├── faces_ms1m_refine_v2_112x112-00002.tfrecord  
├── ...  
└── faces_ms1m_refine_v2_112x112-00099.tfrecord
```

Recommended Dataset: MS1M-ArcFace

- Total Images: 5,822,653
- Identities: 85,742 classes
- Images per Identity: ~68 average (range: 10-500)
- Original Image Size: 112×112 pixels (JPEG format)
- File Format: TFRecord (100 shards)
- Total Size: ~42GB
- Quality: Cleaned and refined version of MS-Celeb-1M

NOTE: please check training data set license for commercial use

Image Specifications

- Format: JPEG-encoded images
- Original Size: 112×112 pixels
- Color Space: RGB (3 channels)
- Alignment: Faces should be centered and aligned
- Quality: High-quality face crops with minimal occlusion

Data Preprocessing

The training pipeline automatically applies:



1. JPEG Decoding: Decode compressed JPEG images
2. Resizing: Resize from 112×112 to targeted size of 96×96
3. Type Conversion: Convert to float32
4. Normalization: $(\text{pixel} - 127.5) \times 0.0078125$ (range: [-1, 1])
5. Augmentation: Random horizontal flip (50% probability)
6. Label Encoding: Convert integer labels to one-shot encoding

iii. Training Modes

Standard Mode (Default)

- Description: Efficient data loading using TensorFlow's 'tf.data' API
- Memory Usage: Low - loads batches on-the-fly
- Performance: Optimized with prefetching and parallel processing
- Best For: All dataset sizes, especially large datasets (>1M images)
- Enable: Default behavior (no flag required)

Features:

- Parallel TFRecord reading (num_parallel_reads=AUTOTUNE)
- Parallel preprocessing (num_parallel_calls=AUTOTUNE)
- Data prefetching for GPU utilization
- Automatic batching and shuffling
- Infinite dataset repetition for epoch-based training

Mixed Precision Training (Optional)

- Description: Uses FP16 (half-precision) for faster training
- Enable: --mixed_precision flag

iv. Command-Line Argument

Dataset Arguments

- --data_path (string, required)
Path pattern to TFRecord files containing training data.
 - Format: Glob pattern (e.g., /path/to/dataset/*.tfrecord)
 - Example: /home/user/faces-ms1m/*.tfrecord
- --num_classes (integer, required)
Number of identity classes in the dataset.



- Range: 1,000 to 1,000,000+
- Default: 85742 (MS1M-ArcFace dataset)
- Note: Must match the number of unique identities in your dataset
- --input_size (integer, optional)
Input image size (square). Images will be resized to input_size × input_size.
 - Options: 96 or 112
 - Default: 96

Training Hyperparameters

- --batch_size (integer, optional)
Number of samples per training batch.
 - Range: 32 to 1024 (GPU memory dependent)
 - Default: 512
- --epochs (integer, optional)
Maximum number of training epochs.
 - Range: 50 to 200
 - Default: 100
- --steps_per_epoch (integer, optional)
Number of training steps (batches) per epoch.
 - Calculation: $\text{total_samples} / \text{batch_size}$
 - Default: 11372 (for MS1M with batch_size=512)
 - Example: For 5.8M samples with batch_size=256: $5822653 / 256 = 22745$
- --initial_lr (float, optional)
Initial learning rate for the optimizer.
 - Range: 0.001 to 0.1
 - Default: 0.01
 - Schedule: Automatically reduced at epochs 25, 50, 75
 - Epochs 0-24: 0.01
 - Epochs 25-49: 0.001
 - Epochs 50-74: 0.0001
 - Epochs 75+: 0.0001



- `--weight_decay` (float, optional)
L2 regularization weight decay applied to all convolutional layers.
 - Range: 0.0001 to 0.001
 - Default: 0.0005
 - Purpose: Prevents overfitting by penalizing large weights

Model Arguments

- `--mixed_precision` (flag, optional)
Enable mixed precision (FP16) training for faster training and lower memory usage.
 - Default: Disabled
 - Benefit: ~40% memory reduction, ~2× training speedup

Checkpoint Arguments

- `--checkpoint_dir` (string, optional)
Directory where model checkpoints will be saved.
 - Default: `./ckpt`
 - Contents:
 - per-epoch checkpoints
 - `final_model.keras` (final trained model)
 - `exported_models/` (exported formats if enabled)
- `--save_best_only` (flag, optional)
Save only the best model based on training loss instead of saving every epoch.
 - Default: Disabled (saves every epoch)
 - Benefit: Saves disk space
 - Drawback: Cannot resume from intermediate epochs

Logging Arguments

- `--log_dir` (string, optional)
Directory for TensorBoard logs.
 - Default: `./logs`
 - Contents: TensorBoard event files for visualization



- `--tensorboard` (flag, optional)
Enable TensorBoard logging for training visualization.
 - Default: Disabled
 - Logged Metrics: Training loss, accuracy, learning rate schedule, model graph

View logs in another terminal:

`tensorboard --logdir ./logs`

- `--verbose` (flag, optional)
Print detailed model summary before training starts.
 - Default: Disabled
 - Output: Layer-by-layer architecture, parameter counts, output shapes

Early Stopping Arguments

- `--early_stopping` (flag, optional)
Enable early stopping to halt training when validation loss stops improving.
 - Default: Disabled
 - Benefit: Prevents overfitting and saves training time
- `--patience` (integer, optional)
Number of epochs with no improvement before stopping training.
 - Range: 5 to 50
 - Default: 10
 - Only Used: When `--early_stopping` is enabled

Export Arguments

- `--export_pb` (flag, optional)
Export trained model as frozen TensorFlow protobuf (.pb) format.
 - Default: Disabled
 - Output: `frozen_pb/mobilefacenet_96_96_frozen.pb`
- `--export_float32` (flag, optional)
Export trained model as Float32 TensorFlow Lite (.tflite) format.



- Default: Disabled
- Output: mobilefacenet_96_96_float32.tflite
- --export_int8 (flag, optional)
Export trained model as INT8 quantized TensorFlow Lite (.tflite) format.
 - Default: Disabled
 - Output: mobilefacenet_96_96_int8_quantized.tflite
- --export_all (flag, optional)
Export all model formats (frozen PB, float32 TFLite, INT8 TFLite).
 - Default: Disabled
 - Equivalent To: --export_pb --export_float32 --export_int8
- --num_calibration_samples (integer, optional)
Number of calibration samples for INT8 quantization.
 - Range: 50 to 1000
 - Default: 100
 - Only Used: When --export_int8 or --export_all is enabled

v. Training Examples

Basic Training

Minimal command for standard training:

```
$ bash  
$ python train.py
```

Training with Custom Dataset

```
$ bash  
$ python train.py \  
  --data_path "/data/custom_faces/*.tfrecord" \  
  --num_classes 10000 \  
  --input_size 112 \  
  --batch_size 256 \  
  --epochs 80
```

Training with Model Export

Train and export all model formats:

```
$ bash  
$ python train.py \  
  --batch_size 512 \  
  --epochs 100 \
```



```
--export_all \  
--num_calibration_samples 200
```

vi. Evaluation Metrics

- Loss: ArcFace loss with angular margin (lower is better, target: < 3.0 after 100 epochs)
- TAR-over-FAR: measures a model's true accept rate at a fixed false accept rate ($1e-5$), and this metric is commonly evaluated on the LFW face-verification benchmark

vii. Annotation particularities

- Identity-only labels: Each face image has an integer ID corresponding to a specific person.
- Faces already cropped & aligned: The version used in MSIM-Refine-V2 is the cropped and aligned MS-Celeb-1M used in InsightFace pipelines.
- No extra metadata: There are no bounding boxes, landmarks, gender, age, or pose annotations



5. References

1. Antispoofing

Dataset:

- [CelebA Spoof For Face AntiSpoofing](#)

Papers:

- [1710.07557](#)

2. SCRFD

- [\[2105.04714\] Sample and Computation Redistribution for Efficient Face Detection](#)

Models:

- [insightface/detection/scrfd at master · deepinsight/insightface · GitHub](#)

Datasets:

- [WIDER FACE: A Face Detection Benchmark](#)

3. MobileFaceNet

Papers:

- <https://arxiv.org/abs/1804.07573>

Models:

- https://github.com/sirius-ai/MobileFaceNet_TF
- <https://www.kaggle.com/code/jasonhchwong/mobilefacenet/>

Datasets:

- <https://exposing.ai/msceleb/>
- <https://www.kaggle.com/datasets/jasonhchwong/faces-ms1m-refine-v2-112x112-tfrecord>